

Verilog 4-bit 計數器實作指南

這份指南涵蓋了您在 VS Code 中撰寫、編譯及模擬 4-bit 計數器所需的程式碼與指令。

步驟 1: counter.v (電路主體)

這是一個同步計數器，在每次時脈上升沿 (posedge clk) 觸發。

```
module counter (  
    input wire clk,  
    input wire reset,  
    output reg [3:0] count  
);  
  
    always @(posedge clk) begin  
        if (reset)  
            count <= 4'b0000;  
        else  
            count <= count + 1;  
        end  
  
endmodule
```

步驟 2: tb_counter.v (測試平台)

測試平台產生時脈訊號、控制重置訊號，並使用 \$dumpfile 將波形輸出以供 GTKWave 檢視。

```
module tb_counter;  
    reg clk;  
    reg reset;  
    wire [3:0] count;  
  
    counter uut (  
        .clk(clk),
```

```

        .reset(reset),
        .count(count)
    );

    always #5 clk = ~clk; // 產生週期為 10 的時脈

    initial begin
        $dumpfile("wave.vcd"); // 產生波形檔
        $dumpvars(0, tb_counter);

        clk = 0;
        reset = 1;
        #15 reset = 0; // 經過 15 單位時間後釋放重置
        #100;          // 模擬運行 100 單位時間
        $finish;
    end

    // 在終端機印出文字除錯資訊
    initial begin
        $monitor("Time=%0t | reset=%b | count=%b", $time, reset, count);
    end
endmodule

```

步驟 3: 終端機執行指令

在 VS Code 的終端機依序輸入以下指令：

1. 編譯: `iverilog -o sim_out counter.v tb_counter.v`
2. 模擬: `vvp sim_out`
3. 觀看波形: `gtkwave wave.vcd`